

负载并不是你应该平衡的：引入 Prequal

Bartek Wydrowski
Google Research

Robert Kleinberg
Google Research

Stephen M. Rumble
Google (YouTube)

Aaron Archer
Google Research

摘要

我们提出 Prequal (Probing to Reduce Queuing and Latency), 这是一种用于分布式多租户系统的负载均衡器。Prequal 在服务器能力异构且对抗性负载非均匀、随时间变化的情况下, 旨在最小化实时请求延迟。它通过主动探测服务器负载, 利用 power of d choices 模式, 并在此基础上扩展了异步和可重用的探测机制。与传统观念相反, Prequal 并不均衡 CPU 负载, 而是根据估计的延迟和在途活跃请求 (RIF) 来选择服务器。我们在一个测试平台系统上探索了其设计特征, 并在 YouTube 上进行了评估, 该系统已部署超过两年。Prequal 显著降低了尾部延迟、错误率和资源使用, 使 YouTube 及 Google 的其他生产系统能够以更高的利用率运行。

1 引言

我们报告了将 d 选择 (PodC) 负载均衡范式 [1,2,18] 部署至 Google 运行多个大规模网络服务的经验, 重点关注 YouTube, 该系统已成功运行两年。据我们所知, 这是首次公开报道 PodC 在此规模下成功用于负载均衡。

PodC 范式涉及对 $d \geq 2$ 个服务器进行采样以获取其负载, 并将下一个请求发送到负载最低的服务器。任何实现中的两个核心问题是: (1) 使用何种信号来表示负载, 以及 (2) 如何进行采样? 我们通过名为“探测以减少队列和延迟 (Prequal)”的负载均衡系统回答了这些问题。具体而言, 我们使用两个信号: 飞行中请求数 (RIF) 和延迟, 并通过主动探测的方式来采样服务器。

许多现有的负载均衡系统 (例如, NGINX [19]、Envoy [7]、Finagle [8]、YARP [26]、C3 [23]) 提供某种形式的 PodC, 其中大多数系统将 RIF、延迟或两者的组合

作为负载均衡信号。我们引入了两项主要的新创新。首先, 我们以一种特别有效的新方式结合了 RIF 和延迟, 称为 热-冷字典序 (HCL) 规则。其次, 我们引入了一种新颖的异步探测机制, 在保持同步探测所提供的负载信号新鲜度的同时, 降低了探测开销 (CPU + 关键路径延迟)。

为了本文的目的, 我们将 Prequal 称为一种负载均衡系统, 但从技术上讲, 它实际上是 Google 的 Stubby RPC 框架 (外部称为 gRPC [24]) 中实现的一种负载均衡策略。因此, 我们期望我们的某些创新能够被集成到其他现有的负载均衡器中。我们的主要重点是上述两项创新 (HCL、异步探测); 我们系统中的其他实现细节与我们的核心观点无关。

过去两年中, Prequal 已在谷歌超过 20 个大规模服务中部署。除了驱动大部分 YouTube 服务栈外, 它还在多种其他应用中取得了成功, 包括搜索广告、日志处理以及机器学习模型服务。在典型的应用场景中, 查询处理时间约为 $O(10 - 100)$ 毫秒, 但在某个系统中, 查询耗时长达 $O(10)$ 分钟, 而在另一个系统中, 查询时间则从几秒到数小时不等。这些服务大多属于相互调用的复杂服务网络, 我们发现, 无论网络中的其他服务是否也使用 Prequal, 为最关键的几个服务部署 Prequal 均带来了显著收益。

我们关注探测带来的 CPU 开销和延迟, 但在我们的部署经验中发现这些开销很小, 并且能够带来更大的收益。在数据中心内部, 探测响应时间远低于 1 毫秒。对于某些应用, 毫秒的几分之一也至关重要, 因此我们采用了异步探测, 将其从关键路径上移除。我们创建了可任意缩小 CPU 开销的参数。此外, 更好的资源均衡通过减少资源竞争节省了 CPU。总体而言, 我们发现部

署 Prequal 后, CPU 利用率通常会略微下降。更重要的是, 系统在峰值负载时会进行资源预留以控制尾部延迟, 因此即使平均 CPU 使用率略有上升, 结合尾部延迟的降低, 仍能带来资源预留方面的收益。

我们的对 Prequal 的评估包含两部分: 在 YouTube 上的野外整体评估 (§2 和 §3), 以及在测试平台上对各个主要设计选择进行独立评估的基准测试 (§5)。我们在 YouTube 及 Google 其他部分所取代的默认现成负载均衡策略是 (动态) 加权轮询 (WRR), 该策略专注于在单个任务中的分布式服务器之间平衡 CPU 使用率。因此, §3 中的评估重点在于将 Prequal 与 WRR 进行对比。§2 解释了为何像 WRR 这样的 CPU 平衡策略在此环境中是强有力的竞争对手, 但也说明了重新思考这一方法的重要性, 从而引出了我们论文的标题。§5 和 §6 包含了与文献中知名负载均衡策略的定量和定性比较, 包括 PodC 的其他变体。

一个大规模的网络服务, 如 YouTube, 包含众多子任务——回答搜索查询、提供推荐以及投放广告等——每秒处理海量查询。高效且可靠地管理数据中心资源以运行此类服务是一项重大挑战。为了避免过度超配资源, 系统需要具备灵活性, 允许某些任务在同台机器上其他任务处于低利用率状态时, 暂时超出其分配的资源上限。然而, 这与任务间的性能隔离相矛盾, 危及系统可靠地为每个查询提供低延迟响应的能力。

在本文中, 我们主张, 在诸如大规模网络服务这类系统中, 多个客户端各自自由独立的负载均衡器管理, 共享同一组服务器池, 此时资源配置效率与服务可靠性之间的矛盾, 要求重新思考负载均衡的范式。特别是, 在此类系统中, 负载均衡器的目标不应是将客户端的负载均匀分配到所有可用服务器上。最终目标是实现高容量下的可靠低延迟, 因此负载均衡器应致力于最小化尾部延迟, 并使用与该目标一致的指标进行评估。

在本研究中, 我们提出探查以降低排队与延迟 (Prequal), 一种基于主动探查可用服务器以监控其延迟和正在进行的请求数量 (RIF) 的负载均衡器。Prequal 采用 d 个选择的威力范式 [1, 2, 18], 避免选择过载的服务器。它在基本范式基础上引入了异步且可复用的探查机制, 并支持在优先选择低延迟服务器与选择请求较少的服务器之间进行可定制的权衡。

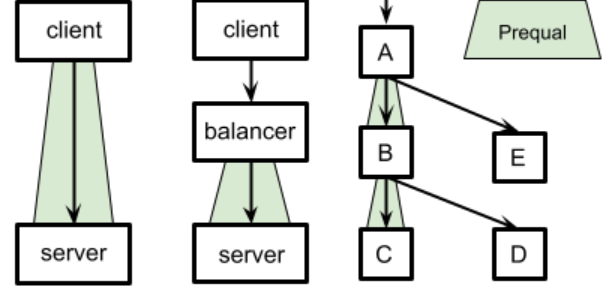


图 1: 查询沿树向下流动, 响应沿树向上流动。每个链路上的分离平衡任务是可选的。负载均衡策略可因链路而异: $A \rightarrow B$ 使用预筛选, 而 $B \rightarrow D$ 不使用。

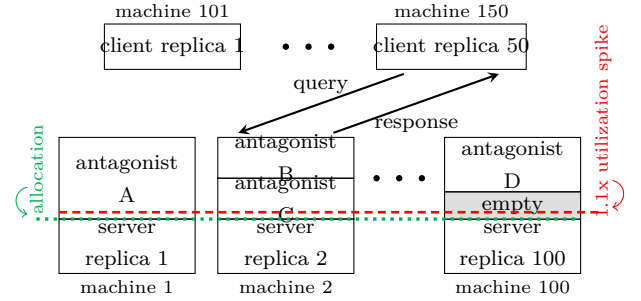


图 2: 聚焦任意一条链路从 Fig. 1: 服务器副本在客户端副本发出查询时响应, 同时其所在机器上运行着对抗性进程。在需求激增期间, 总利用率 (虚线) 可能超过分配量 (点线)。

2 环境和 动机

我们描述了 YouTube 及其他大型云服务的运行环境, 说明为何平衡 CPU 负载似乎是一种显而易见的更优方法。接下来, 我们分析一个平衡 CPU 负载反而导致严重问题的场景, 从而引出 Prequal 方法。最后, 我们展示了来自 YouTube 的数据, 表明我们所提出的动机情况在实际中十分常见。

大规模服务如 YouTube 通常由许多分布式任务组成, 这些任务通过 RPC 相互发起查询。Fig. 1 描绘了一个极度简化的版本, 仅有五个任务, 查询沿树状结构向下流动 (箭头所示), 而响应则反向向上返回 (与箭头方向相反)。在任意一次交互中, 发送查询的任务称为客户端任务, 而返回响应的任务称为服务端任务。例如, 在边 $A \rightarrow B$ 上, A 是客户端, B 是服务端; 而在边 $B \rightarrow C$ 和 $B \rightarrow D$ 上, B 则是客户端。为确保可扩展性和冗余性, 每个任务都会分布在大量机器上, 这些机器被

称为副本 (Fig. 2)。为了避免任何服务端副本过载，每对通信任务之间必须使用负载均衡策略，以决定每个查询应由哪个服务端副本接收。某些策略需要通过一个分离的负载均衡任务来代理查询，该任务唯一功能是决定每个查询应转发到何处，而其他策略则可以直接在客户端副本和服务端副本之间运行 (Fig. 1)。¹ Prequal 适用于这两种运行模式，事实上我们在谷歌的不同系统中均以两种方式使用它。

每个任务运行于单个数据中心内，尽管不同任务可能运行在不同的数据中心。每个副本在其自身的虚拟机 (VM) 内运行，该虚拟机拥有其宿主主机上保证的一定比例的 CPU 周期；这一数量称为其 (CPU) 分配。在任意给定时间段内，副本实际使用的分配比例称为其 (CPU) 使用率，该值可能高于或低于 100%，因为分配仅是一个保证的最小值。服务端任务的聚合（任务）使用率是指其所有副本在整个任务的总 CPU 分配上所使用的比例。通常情况下，每个副本具有相同的分配，因此聚合任务使用率等于平均副本使用率。

一个服务器副本通常与许多其他虚拟机共享其所在机器，我们称这些虚拟机为竞争者。机器 (CPU) 利用率既包括我们的服务器副本，也包括所有竞争者，它不同于单个副本和作业的利用率。为了确保多租户系统中所有用户的一致性能，隔离机制试图防止一台机器上的某个虚拟机的行为对另一台虚拟机的行为产生不利影响。这里的理念是：“如果你的使用量保持在分配范围内，你就不会有问题。”然而，如果某个虚拟机上溢了其 CPU 分配，当隔离系统启动时，它可能会受到影响。

在该隔离机制和设计哲学下，试图在单个作业内的副本之间均衡 CPU 利用率似乎是一种不言而喻的最佳策略。² 事实上，加权轮询 (WRR) 策略在 Google 运行良好。它利用各副本近期的良好吞吐量 (goodput)、CPU 利用率及错误率的平滑历史统计量，周期性地计算每个副本的独立权重。客户端随后按这些权重的比例将查询路由至相应副本。在无错误发生的情况下，每个副本的权重 w_i 计算公式为 q_i/u_i ，其中 q_i 和 u_i 分别表示副本 i 最近的每秒查询数 (QPS) 速率与 CPU 利用率。历史上，WRR 最初源于网络领域 [11]，但亦已被适配用于如前所述的副本选择 [4, 9]。

接下来，我们描述一种即使能完美均衡 CPU 负载，该方法仍会适得其反的情形。假设我们的服务器作业在

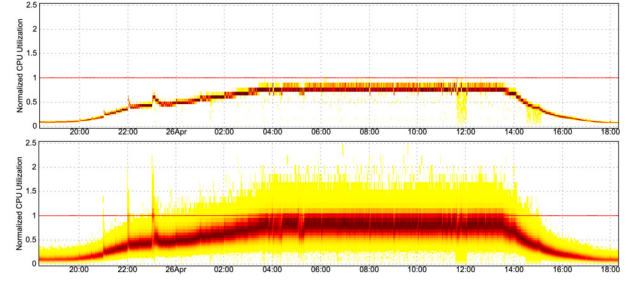


图 3: 规范化的 CPU 使用率热力图，采样间隔为 1 分钟 (上) 和 1 秒 (下)，展示的是在单个数据中心内数百台机器上运行的 YouTube 主页服务。颜色越深表示该区域越密集。使用上限由 1.0 (红线) 表示，在所有 1 分钟间隔内均满足要求，但在 1 秒间隔下，高峰负载时经常被突破——有时甚至超过两倍！

完全相同的机器上运行 100 个副本，每个副本在其所在机器上被分配了 40% 的 CPU。进一步假设：对抗性负载 (antagonist load) 已占满第 1 和第 2 台机器剩余的全部 60% CPU 资源，而其余 98 台机器则拥有充足的空闲容量，安全地高于 4%。最后，假设我们的作业遭遇一次临时性的需求激增，使其整体 CPU 利用率平均提升至其分配额度的 1.1 倍，即每台机器平均达 44%。若负载均衡器试图在所有副本间均衡 CPU 利用率，则会力求使每个副本均达到这一平均值。此时全部 100 个副本都将超出其 CPU 分配额度；但后 98 个副本尚可接受，因为系统允许它们短暂突破配额，以吸收未被使用的 CPU 周期。然而，在竞争激烈的第 1 和第 2 台机器上，CPU 隔离机制通常会被触发，从而严重限制这些副本的性能，有时甚至影响其服务的全部查询。因此，尽管问题负载仅出现在我们 100 个副本中 2% 的那两个副本上、且仅超出其配额的额外 10% (即约占我们整体 1.1 倍负载的 $\sim 0.18\%$)，却可能导致我们全部查询中整整 2% 的查询性能下降。p99 尾部延迟³ 很可能出现尖峰。Fig. 6 在 §5.1 中在测试平台上表现出这种行为。

在这种过载情况下，均衡副本利用率恰好是错误的负载均衡策略，因为可用机器吸收额外负载的能力差异很大。此外，由于可用容量的差异是由对立过程引起的，无法提前由我们的任务预测，但可能在运行时被检测到。

均衡副本利用率在所有副本始终处于其分配范围内时是一个不错的主意，因为这能在分配限制下最大化我们所能处理的流量。然而，Fig. 3 显示我们很容易被误

¹在此规模下，即使平衡任务本身也必须分布在多台机器上。

²在全部这些异构机器之间均衡机器 CPU 利用率并无意义。

³我们使用 p99 之类的表达式来表示分布的百分位数。

导。以 1 分钟为时间间隔绘制 CPU 利用率，会让我们以为各个副本都遵守了其分配限额，但使用 1 秒的时间间隔则揭示出信号中更大的潜在波动性，频繁出现接近两倍于上限的突发！换句话说，过载并非一种特殊情况；在足够小的时间尺度下，几乎总会有某个副本处于过载状态，即使整体负载仍在作业分配范围内。唯一的问题是，这个副本是否不幸地运行在高度竞争的机器上，以及该峰值持续时间是否足以触发隔离机制。

因此，避免高尾部延迟需要某种机制，能够实时快速地向客户端发出警告，提示负载过重的副本。该机制应使用尽可能最新的负载信号，并且在预测未来请求服务时的高延迟方面具有高度前瞻性。CPU 利用率无法满足这些要求，部分原因在于它是一个滞后信号。为了使它具有意义，必须在一段时间窗口内进行平均，这自动导致其过时性存在下限。此外，该信号还忽略了其他影响延迟的因素，例如锁争用、内存带宽或其它共享资源的竞争，而这些因素有时难以测量或隔离。

Prequal instead 使用两个负载信号：RIF 和延迟。RIF 是一个瞬时信号（即，在探测时刻其精确值即可获得），而 Prequal 所使用的延迟估计也几乎是瞬时的 (§4)。以下是我们的主要设计目标。

1. 最小化探测开销。每次查询的探测次数应为一个较小的 $O(1)$ 且延迟估计算法（在服务器副本上运行）必须轻量，每次查询的更新时间为 $O(1)$ 或 $\tilde{O}(1)$ 。
2. 探测不应给查询的关键路径带来显著延迟。Prequal 通过异步探测实现这一点：当前查询的分配利用由先前查询发起的探测响应。
3. 降低尾部延迟。Prequal 在用于副本选择之前，会移除最差的探测（即 RIF 和/或估计延迟最高的探测），这一过程受到带有记忆的平衡分配理论的启发 [14, 16]。
4. 限制服务器副本上查询处理的内存占用。⁴ Prequal 会避免将查询分配给具有异常高 RIF 的副本，即使它们的估计延迟较低。RIF 信号具有双重作用，因为它也是未来负载的强有力先行指标。

回想一下，Prequal 可以在有或没有专用负载均衡层 (Fig. 1) 的情况下使用。专用层的优点包括：(1) 当

⁴RIF 对 RAM 有重大影响，因为大多数飞行中的查询都处于某种处理阶段，而不是停留在队列中。RPC 本身通常都很小。

客户端查询远端数据中心的服务器任务时，能够保持探测请求的本地性；(2) 负载均衡器通常比客户端拥有的副本少，因此每个客户端看到的查询流比例更大，其探测结果也更及时（以自最近一次探测以来落在服务器副本上的查询数量来衡量）；(3) 对负载均衡器进行软件升级无需影响客户端。缺点包括：(1) 增加了额外的 RPC 开销（延迟、CPU、网络开销）；(2) 需要额外管理一个任务；(3) 仍然需要对到达负载均衡器的负载进行平衡，尽管这通常更容易，因为负载均衡器的工作量较为均匀，且通常通过过度配置负载均衡器而非服务器来降低成本。

3 YouTube 上的部署评估

几年前，我们确定负载不平衡是导致持续违反 SLO 的原因。⁵ YouTube 中的违规行为，促使我们将其部分关键服务切换至 Prequal。这些服务包括负责用户个性化推荐的 YouTube 首页服务。采用 Prequal 后，我们在各项指标上均实现了显著提升，包括尾部延迟降低 2 倍、尾部请求注入频率 (RIF) 降低 5-10 倍、尾部内存使用率降低 10-20%、尾部 CPU 利用率提高 2 倍，并几乎完全消除了因负载不均导致的错误。除了满足服务级别目标 (SLOs) 外，这还使我们能够显著提高各数据中心可接受流量的目标值，从而节省了大量资源。基于这一成功经验，我们将 Prequal 推广至 YouTube 的众多其他核心服务，并随后推广至 Google 内部的各类其他服务，具体详情见 §1。

YouTube 服务由众多子任务组成——回答搜索查询、提供推荐以及展示广告等。其中一些子任务的查询会触发级联的子查询，如 Fig. 1 所示。由于这些任务类型差异巨大，其查询处理需求 (CPU、内存、延迟等) 也各不相同。在本节中，我们通过使用主页任务自身的一小部分实时用户流量，将调度策略从加权轮询 (WRR) 切换为预筛选 (Prequal)，并保持系统其他部分的负载均衡策略不变。历史上，主页任务是首个部署 Prequal 的服务器任务，而大多数其他任务仍使用 WRR。在本实验进行时，大多数但并非全部任务已开始使用 Prequal。两种情况下，切换带来的影响相似。我们在此陈述 Prequal 的影响，但其设计细节将推迟至 §4 进行说明。随后，我们通过在测试平台（非 YouTube）上的实验来探索这一设计空间，详见 §5。

⁵SLO = 服务等级目标 [10]

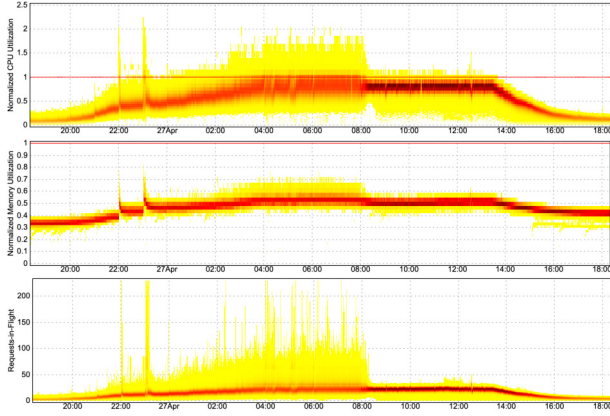


图 4: 每个 YouTube 首页服务器副本的规范化的 CPU 使用率、规范化的内存使用率以及在途请求数量的热力图, 首先使用加权随机法 (WRR) 进行负载均衡, 随后在 08:00 后不久切换至 Prequal。

注意到的第一个结果是, 显式地在 RIF 上进行平衡确实有效, 将尾部从大约 225 降低到大约 50 (Fig. 4)。由于主页查询处理携带了大量的每查询状态, 这使得我们的尾部内存使用量降低了 10-20%, 从而相应地减少了我们的内存占用。如图 3 所示, WRR 在 1 分钟时间尺度上非常有效地维持了紧密的 CPU 负载分布, 但在 1 秒分辨率下测量尾部 CPU 利用率时表现较差。Prequal 解决了这个问题, 使尾部降低了约 2 倍。因此, Prequal 将偶尔的服务器副本错误尖峰从 0.01-0.1% 降至几乎为零, 并将中位数延迟降低了 10-20%, 尾部延迟降低了 40-50% (Fig. 5)。

Fig. 5 分别对每个延迟分位数 (p50、p99、p99.9) 进行归一化, 基于每日流量低谷时该分位数的典型值。结果显示, Prequal 在高峰负载下将尾部延迟显著降低, 导致 p99 和 p99.9 的延迟在乘法意义上反而比 p50 的恶化程度更小。这与通常预期的行为相反, 而我们对 WRR 确实观察到了正常的情况。

在这些实验中, 我们将 Prequal 配置为每查询发送 5 个探测包, 这意味着 RPC 总数乘以 6。此外, 作为 Prequal 的早期采用者, 该服务仍使用同步探测模式 (§4), 这增加了关键路径的延迟。尽管我们计划迁移到异步探测, 但根据我们在 YouTube 的经验, 通过拉取尾部数据所获得的收益远超过这些开销。对于请求相对较重的服务尤其如此, 例如 $O(10-100)$ milliseconds 的每查询处理时间。我们为探测设置了 3ms 的超时时间, 但大多数

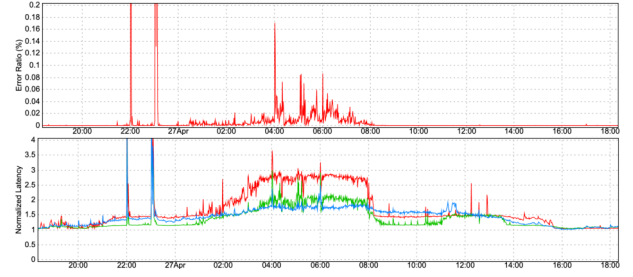


图 5: 规范化的 YouTube 主页服务器请求错误率和延迟的 p99.9 (红色)、p99 (绿色) 和 p50 (蓝色)。故障是由于超时或负载削峰导致的不平衡所引起。在 08:00 左右从加权随机路由 (WRR) 切换到预筛选 (Prequal) 后, 大部分错误被消除, 尾部延迟降低了 40-50%, 中位延迟降低了 5-10%。每个延迟分位数均分别规范化为每日低谷时段 该分位数的典型值, 因此在切换后 p50 曲线可能高于另外两条曲线。22:00 和 23:00 附近的尖峰与本实验无关。

探测返回时间远快于该值。⁶ 探测延迟以及处理探测所额外消耗的 CPU 时间, 与节省的时间相比都属于噪声。

4 系统设计

当对查询洪流进行负载均衡时, 每一毫秒都至关重要。Prequal 的设计使得客户端能够基于最新信息 (理想情况下不超过几毫秒) 选择一个副本, 同时将发送和接收探测请求的过程排除在关键服务路径之外。因此, 最关键的设计决策集中在如何维护一组高质量的探测请求, 以及如何根据探测池中的信息将查询分配给 副本。

以下是一些关键问题。

1. 客户端应多久探测一次副本, 以及应探测哪些副本?
2. replicas 在探测响应中应报告哪些信息, 以及如何计算这些信息?
3. 在选择用于处理查询的副本时, 客户端应如何利用其探测池中的信息?
4. 客户端应如何管理其探测响应池? 特别是, 应在何时从池中移除元素?

探测速率 客户端会发出指定数量的探测请求, r_{probe} , 这些探测请求由每个查询触发。此外, 还可以配置客户

⁶在谷歌的其他地方, 我们成功地使用了 1 毫秒的超时设置, 这里可能也可以做到。

端在超过指定的最大空闲时间后发出探测请求，以确保即使在过去一段时间内没有查询到达，探测响应池中仍能保持最新的探测结果。因此，探测速率（单位时间内的探测次数）等于查询到达率与 r_{probe} 的乘积（或极小探测速率，取较大者）。我们将探测速率与查询速率关联起来，因为这是客户端必须做出决策的速率。它也与副本上的负载统计量（尤其是 RIF）随时间变化的快慢密切相关，尽管这还取决于客户端副本与服务器副本的比例。如果探测速率远高于查询到达速率，则会产生大量冗余探测；而探测速率过低则存在风险，即探测结果在被使用时可能已“过期”（与副本当前状态的相关性较弱）。

探测目标从可用副本集合中以均匀随机的方式无放回采样。这一设计选择受到平衡分配理论文献的启发，该文献主张对探测目标进行均匀随机采样。此外，这也有助于避免“雷鸣群集”现象，即当一个估计延迟较低的副本同时被多个客户端视为最佳选择时，会遭受大量查询的冲击，导致队列堆积和高延迟 [15]。在 Prequal 中，这种情况不太可能发生，因为每个客户端的探测池仅代表副本的一个小规模随机子集，反之，每个副本也仅属于一小部分（期望意义上）随机客户端的探测池。

我们允许 r_{probe} 为分数⁷，甚至少于一个。探测会消耗少量但非零的服务器 CPU 和网络带宽，因此希望将探测率降低到对尾延迟影响可以忽略的程度。我们的实验在 §5.3 中表明， $r_{\text{probe}} > 1$ 从尾延迟的角度来看非常安全。

负载信号 Prequal 包含一个服务器端模块，用于跟踪 RIF 和延迟统计量，并响应探测请求，具体如下。我们称查询在应用程序逻辑接收到 Stubby 的 RPC 时“到达”服务器，而在应用程序逻辑将响应 RPC 返回给 Stubby 时“完成”。我们将查询的延迟定义为该时间段的长度，在此期间，查询会贡献于该服务器的 RIF 计数。如果存在应用层队列，则延迟包括在队列中的停留时间。然而，更常见的情况是，应用程序避免使用队列，而是依赖线程或协程调度。我们不尝试捕捉网络延迟，因为所有服务器副本都位于同一数据中心内。

在响应探测请求时，RIF 仅通过检查一个计数器即可获得。而延迟估计则更为复杂。当查询完成时，我们会记录其延迟值，并以查询到达时的 RIF 计数作为标签。

⁷每个查询会触发 $\lceil r_{\text{probe}} \rceil$ 或 $\lceil r_{\text{probe}} \rceil$ 探测，通过确定性舍入，以保证在极限情况下每查询有 r_{probe} 次探测。

当探测请求我们估算延迟时，我们会查阅当前（或附近）RIF 值下的最近一组延迟值，并报告其中位数。维护这些数据结构的单个查询开销很小。在中等至高查询到达率下，样本足够丰富，因此我们完全基于过去 $O(1-10)$ 毫秒内完成的查询来计算延迟估计。

探测器池 预资格客户维护一个探针响应池，用于副本选择。池中的每个元素表示响应的副本以及接收到响应的的时间。⁸ 以及上述讨论的负载信号。该池的最大大小受到限制：我们发现，16 的池大小已足以实现 Prequal 的优势，而超过 16 后的增益则较为有限。在接收到新的探测响应时，会将其添加到池中，如有必要则淘汰一个探测以遵守最大池大小的限制。

复制选择 Prequal 可以访问许多负载信号，我们选择重点关注 RIF 和延迟。延迟信号非常明显：既然我们希望最小化延迟，为什么不将查询路由到表现出最低延迟的副本呢？RIF 部分很重要，因为当副本必须存储大量每查询状态（如 YouTube 所示）时，其内存分配必须是常数偏移量加上一个与最大预期 RIF 呈线性关系的项。此外，RIF 是一个瞬时信号，能够提前预示未来的负载，因为它代表了当前正在处理的查询。而延迟信号则基于最近完成查询的观测延迟。这两个信号都比 CPU 使用率更新，因为后者需要长时间平均才有意义。

从内存角度而言，只有当我们选择的副本会提高最大 RIF 时，RIF 才重要。原则上，我们希望在其他任何时候都优先考虑延迟平衡，尽管由于上述领先与滞后指标的考量，我们必须对此倾向加以克制。

为同时最小化延迟和 RIF，Prequal 采用一种热-冷字典序（HCL）规则来选择副本，该规则根据探测的 RIF 值将池中的探测标记为热或冷。Prequal 客户端基于最近的探测响应，维护副本间 RIF 分布的估计。若某池元素的 RIF 超过估计分布的指定分位数 (Q_{RIF})，则将其分类为热，否则为冷。在副本选择中，若池中所有探测均为热，则选择 RIF 最低者；否则，选择延迟最低的冷探测。我们的实验 (§5.2) 表明， $Q_{\text{RIF}} \in [0.6, 0.9]$ 是一个不错的选择，尽管即使取 0 也有效（即仅基于 RIF 的控制）。

如果池为空，则会出现异常情况。此时，Prequal 会简单地退回到选择一个均匀随机的副本。事实上，根据我们的经验，当池的占用率低于 2 时，调用此回退机制是有益的。

⁸发送时间理想，但可能引入时钟偏差。

一种流行的信号组合方法是使用它们的线性组合，并引入一个缩放常数，将它们转换为相同的单位。在我们的实验中，HCL 的表现优于仅使用 RIF 的方法 (§5.3)，而后者又优于所有非平凡的 RIF 与延迟的线性组合 (§5.2和 §A)。HCL 方法还很好地体现了我们的关注层次：控制延迟固然理想，但副本遵守其 RAM 分配通常是一个硬约束。

探针的重复使用与移除 Prequal 通过管理探测池来避免三种情况：状态过期，即负载信号过于陈旧而无法准确反映实际情况；耗尽，即探测池变为空；以及退化，即探测池中所代表的负载呈现出对高负载副本的偏向性选择。这三种情况相互关联，因此讨论不可避免地会有些循环，我们从最简单的情况开始。

耗尽： Prequal 采用多种探针移除机制（如下文详述）来控制过时和退化，包括在探针被使用后将其移除。为了在不增加探测频率的情况下防止池耗尽，我们可以通过将每个探针重复使用最多 b_{reuse} 次来延长其寿命。此重用限制根据以下公式设定

$$b_{\text{reuse}} = \max \left\{ 1, \frac{1+\delta}{(1-m/n) \cdot r_{\text{probe}} - r_{\text{remove}}} \right\}, \quad (1)$$

其中， $\delta > 0$ 是一个配置参数，用于控制探测器在池中累积的净速率， m 是最大池大小， n 是副本数量， r_{probe} 是上文讨论的探测速率， r_{remove} 是下文讨论的探测器移除速率。我们始终设置 $b_{\text{reuse}} \geq 1$ ，当其为分数时，将其随机向下或向上取整，以保持期望值不变。

过时性出现的原因 有两个：老化和过度使用。随着探测器的使用时间增长，其负载数据的准确性会下降，因为副本接收了新的查询并完成了已有的一些查询。过度使用是老化的一种特殊情况，我们可以部分缓解；具体而言，当客户端自身向该副本发送查询时，可以通过增加该探测器的 RIF 值来进行补偿。理想情况下，我们还应提高其延迟估计值，但目前尚未这样做。Prequal 解决一般过时性的方法之一是为探测器设置一个时间限制，并在它们的年龄超过该限制时将其从池中移除。此外，每当有新探测器到达且可能导致池的大小超过其上限时，我们会丢弃最老的探测器。

退化 是这些现象中最微妙的。如果对应于轻负载副本的探测器持续被选中并从池中移除，那么经过多轮副本选择后，留在池中的探测器就对应于高负载的副本。为避免这种情况，Prequal 会周期性地从池中移除最差的探测器。这类似于——但比——标准的 d 随机选取的桶中仅使用负载最少的桶的 d 选择模型要宽松得多。结

果表明，将桶选择策略从“仅使用 d 个随机探测器中的最佳者”扩展为“避免使用 d 个随机探测器中的最差者”，在定性上保持了 d 选择模型的理论保证 [21]。在移除最差探测器时，Prequal 会在两种规则间交替：一是移除最老的探测器（即年龄最差）；二是根据用于副本选择的相同秩序（但顺序相反）来判定最差探测器：如果池中至少有一个热探测器，则移除 RIF 最高的热探测器；否则，移除延迟最高的冷探测器。

我们定义一个 r_{remove} 参数，每次查询时从探测池中删除该数量的探测。与探测速率类似，该数值可以为小数，此时实际删除的探测数量始终为向下取整或向上取整，通过确定性舍入方式，以在平均情况下达到配置的探测删除速率。通过在最旧和负载最高的探测之间交替删除，我们实现了统一的方法，既能避免过时，又能防止性能下降（在探测超时的基础上）。

总而言之，Prequal 会因为四个原因从池中移除探针。它会在必要时驱逐池中最旧的探针，以避免超过最大池大小。当探针达到其重用预算或其年龄超过超时阈值时，它们会被移除。此外，每个查询都会以可配置的速率交替地移除最差和最旧的探针。

同步模式 到目前为止，我们描述了采用异步探测（即异步模式）的 Prequal，该模式维护一个探测池，将探测操作移出关键服务路径。这种异步机制在许多应用场景中非常有用，例如当探测延迟不可忽略，或探测的 CPU 开销相较于查询显著较高时。然而，我们也实现了 Prequal 的同步模式（sync mode），该模式下不使用探测池。具体而言，当一个查询到达时，客户端发出若干 d 个探测（至少 2 个，通常为 3-5 个），向随机副本 () 发起探测，等待接收足够数量的响应（通常为 $d-1$ ），然后依据前述相同的副本选择规则，在这些中进行选择。我们在此介绍同步模式，是因为它已被用于 YouTube 服务（参见文献 §3），但 §5 中报告的所有实验均采用异步 Prequal，因为该模式更适合大多数应用场景。

一种需要同步模式的重要用例是当副本持有影响查询执行成本的状态时，例如，某个副本可能将某些数据缓存在内存中，以避免从较慢的存储层读取。同步探测允许我们将查询的相关信息包含在探测中。如果该副本随后确定由于其缓存中已有的数据，能够更高效地执行该查询，则它可以调整其报告的负载，从而吸引该查询，例如通过将其报告的负载降低 10 倍。我们曾以这种方式在 YouTube 的部分功能中使用同步预判断。

避免错误以防止陷入陷阱 假设某个副本存在某些问题（例如配置错误），导致其对非微不足道比例的查询立即返回错误，从而快速处理这些查询。那么，在剩余成功查询上的延迟（RIF）、CPU 利用率及其他指标将使其看起来比实际接收到的流量所对应的负载要低。如果负载均衡器对此不够智能，该副本可能会吸引越来越多的流量，这种现象被称为 sinkholing。Prequal 包含了一些启发式方法来避免 sinkholing，但由于这些方法并非我们贡献的核心内容，我们选择在阐述中省略细节以简化说明。

5 测试平台评估

在 §3 中，我们报告了在 YouTube 部署 Prequal 的经验。该部分中 Prequal 与 WRR 的对比构成了一种评估类型，在某种程度上代表了真实工作负载的基准定义。然而，对生产环境中部署的 Prequal 进行评估，并不能揭示其设计中各个独立因素对其性能的影响。在本节中，我们报告了一组在测试环境中执行的测试，这些测试允许对 Prequal 及其基准进行更为受控的实验，同时保留了 Prequal 所针对的生产环境中的关键特征：可变的查询成本和不可预测的时间变化对抗负载。

我们首先描述测试平台和基准系统参数。在每个实验中，我们解释了与基准相比所更改的参数。

我们的测试平台包含一个客户端任务和一个服务器端任务，每个任务由多个副本组成，这些副本运行在不同但相同的物理机器上，且位于同一数据中心内。查询代表一种非常简单的 CPU 密集型工作负载：它们只是反复迭代一个开销较大的哈希函数。为了模拟查询成本的差异性，我们改变迭代次数，该数值从一个标准差等于均值的正态分布中抽取，并在零处截断。这些任务在标准的谷歌数据中心内运行，使用的是通用的多核机器，以及谷歌的标准隔离机制，而对抗性流量则仅是我们在实际环境中偶然遇到的流量。

所有实验均使用 100 个客户端副本和 100 个服务器副本，每个服务器副本分配机器 10% 的 CPU。此外，探测池大小 = 16，探测在池中停留一秒钟后过期，Equation (1) 中的网络探测池漂移率为 $\delta = 1$ 。除非另有说明，我们设置 $Q_{\text{RIF}} = 2^{-0.25} \approx 0.84$ 且 $r_{\text{remove}} = 1$ 。我们以每个查询 3 个探测作为基准探测率，以确保探测率不会低到影响性能。根据需要，我们在每个实验中针对不同的总体利用率目标，以引发我们希望展示的行为。

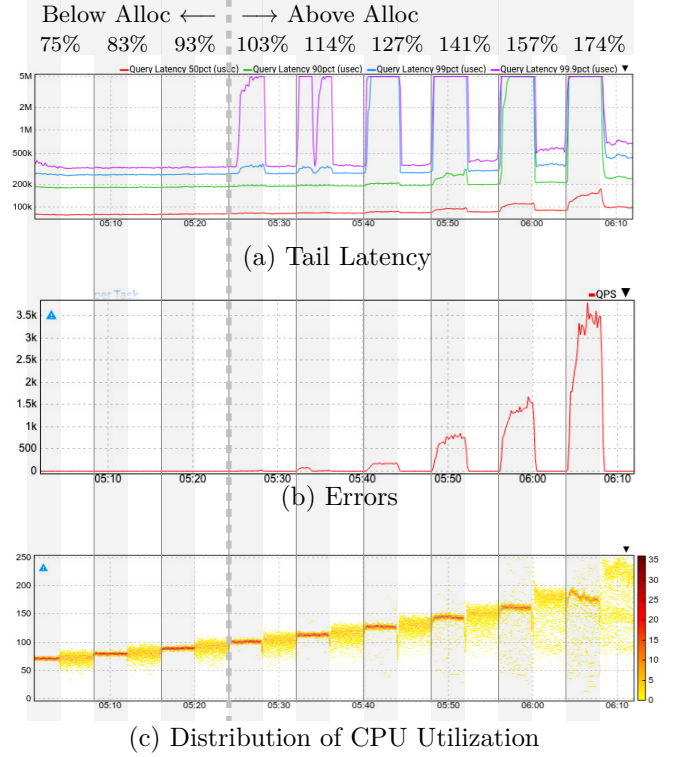


图 6: 负载斜坡实验。灰色背景表示 WRR 策略，白色表示 Prequal。注意尾部延迟采用对数坐标。

无论何时绘制 CPU 利用率，均按分配比例缩放为百分比。

这些实验中的一些图表展示了 RIF 值的分位数。由于这些数据均为整数，分位数也应为整数。然而，当我们的监控系统构建直方图时，所有整数值 k 都会在区间 $[k - \frac{1}{2}, k + \frac{1}{2})$ 内均匀分布。因此，我们 RIF 分位数的图表中包含了小数。

5.1 对可变拮抗负荷的鲁棒性

在本实验中，我们从约 75% 的聚合 CPU 负载开始，以 8 个乘法步骤（每个步骤为 10/9）逐步增加负载，得到 0.75x、0.83x、0.93x、1.03x、1.14x、1.27x、1.41x、1.57x 和 1.74x 的分配值。我们通过增加聚合查询速率（从约 5.6k 查询每秒（以下简称 qps）提升至约 13k qps），同时保持每个查询的平均工作量不变来提高负载。在每个负载级别内，我们前半段时间使用 WRR，后半段时间使用 Prequal。

Fig. 6 中的顶部图表以对数刻度显示了中位数和尾部延迟的影响，单位为微秒。我们为每个查询设置了 5

秒的超时，因此图表在 5 秒处达到上限。在前三个步骤（低于资源分配量）中，WRR 和 Prequal 的延迟分布均保持稳定，约为 80 毫秒（p50）、182 毫秒（p90）、265 毫秒（p99）以及 325 毫秒（p99.9）。随后，在第 4 步首次超出 CPU 分配量（超出 1.03 倍）时，WRR 与 Prequal 的行为迅速发散。

对于 WRR，p99.9 延迟在 5 秒的限制处达到峰值，而 p99 从约 26 到 335 毫秒，略高于之前的 p99.9。到第 6 步（1.27 倍分配），p99 达到峰值，p90 开始出现明显上升，约为 ~203 毫秒（+11%）。到第 8 步（1.57 倍分配），p90 达到峰值，甚至 p50 也上升至 ~111 毫秒（+39%）。到第 9 步，p50 进一步上升至 ~150 毫秒（+88%）。

相比之下，Prequal 在负载步骤 4 和 5 的 p99.9 下几乎没有任何可见变化，到步骤 6 时，p99.9 仅上升至 ~350ms（+8%）。虽然在负载步骤 7（1.41 倍分配）时开始显著上升，但即使到步骤 9 仍维持在约 700ms，远低于 5 秒的超时阈值。与此同时，p99 延迟直到负载步骤 7 才出现明显下降，p90 大约在步骤 8 出现，而 p50 则大约在步骤 9 才出现。

第二幅图显示了错误率，WRR 和 Prequal 在第 3 步之前错误率均为零。注意，此处为每秒的绝对错误数量，而非百分比。WRR 在负载第 4 步开始出现极低的错误率（5 至 20 错误/秒）。至负载第 5 步，错误率明显上升，其增速远超入站查询每秒请求数（qps）。到第 9 步时，超过四分之一的查询返回错误。这些错误几乎全部为“截止时间超限”错误，即查询在 5 秒截止时间内未能完成。相比之下，在本实验的所有负载水平下，Prequal 均返回零错误。

下图显示了 WRR 和 Prequal 的 CPU 利用率分布的热力图。在这里可以看到，WRR 在完成其设计目标方面表现极为出色：均衡 CPU 负载。相比之下，Prequal 的 CPU 分布要松散得多。那么为什么 WRR 的延迟和误差度量如此糟糕呢？这正是我们在 §2 中提出的动机解释。进一步分析 WRR 的服务器级数据（未在图中展示），我们发现高对抗负载的服务器副本与延迟表现较差的副本之间存在强烈的相关系数。而 Prequal 则将负载从这些服务器副本上转移开，以避免最严重的延迟影响。

这一系列结果解释了我们论文标题的由来。尽管对大多数人来说这显得不合直觉，但在此情况下，实现近乎完美负载均衡（WRR）的负载均衡器反而表现更差（错误率和延迟更高）。这是因为负载均衡器的真实目标

并非平衡负载，而是将负载导向具备可用容量的位置。

这些结果之所以成为可能，是因为我们的系统中许多服务器副本并未被对抗者完全分配资源，而且在任意时刻，这些对抗者中的许多并未使用其全部的 CPU 容量。Prequal 允许我们的任务在副本之间动态转移负载，从而利用这些临时的闲置容量。这对服务部署具有两方面的影响：(1) 我们可以更激进地进行资源配置，因为 Prequal 能够应对瞬时负载高峰。(2) 理论上，这甚至可能实现超分配，因为它将相关资源池从单台机器的容量扩展到一个更大的机器集合，从而实现更有效的统计复用。

5.2 复制选择法则

在拥有数千台机器的现代数据中心中，这些机器通常跨越多个硬件代际和处理器架构。在跨多台机器分配任务时，我们可以通过对不同类型的机器应用性能倍数来考虑这些差异，例如，类型 A 的 1 个 CPU 核心相当于类型 B 的 1.2 个 CPU 核心。然而，在大规模场景下做到这一点非常困难，因为性能差异往往高度依赖于具体的工作负载，例如，YouTube 可能更适合某一代硬件，而 Google Maps 则更适合另一代。因此，当我们的副本被分配到具有不同硬件的机器上时，它们的有效吞吐量会变得不一致，而速度较快的机器在无负载时表现出更低的延迟（根据定义）。在这种情况下，降低平均延迟的最佳方法是先将快速机器填满，直到其延迟上升至与慢速机器相当，然后再同时填充快速和慢速机器。

There are various ways that a replica selection rule could be designed to fulfill this goal. We experimented with nine replica selection rules ranging from very simple baselines to sophisticated rules used in popular open-source load balancers (e.g. [26]) and in the research literature (e.g. [23]).

In describing the replica selection rules we evaluated, we distinguish between client-local and server-local quantities. The former are measured at the client (or load balancer) itself, the latter are measured at the server replica and reported back to the client (or load balancer). For example, client-local RIF refers to the number of queries the client has sent to the replica that have not yet yielded responses. Server-local RIF refers to the total number of queries the replica has received

(from all clients) that it has not yet finished processing. In the literature on load balancing of HTTP traffic there is also a distinction between individual requests and connections; the latter may be long-lived, comprising multiple requests. Since the queries in our work represent short-lived connections, we ignore this distinction in our experiment, treating connections and requests as synonymous.

We evaluated the following replica selection rules.

- Random selects a uniformly random replica.
- Round Robin (RR) cycles through the replicas, keeping track of the most recently chosen one and always selecting the next available replica in cyclic order.
- Weighted Round Robin (WRR) is as described in §2.
- Least Loaded (LL) represents the LeastLoaded policy implemented in the NGINX and Envoy reverse proxies [6, 19]. It chooses the available replica with the least client-local RIF, breaking ties in favor of one nearest to the most-recently-chosen replica in cyclic order.
- Least Loaded with Power of Two Choices (LL-Po2C) samples two available replicas uniformly at random and selects the one with the least client-local RIF. This modification of LL is also implemented in NGINX and Envoy.
- YARP-Po2C is a replica selection rule from Microsoft’s YARP reverse proxy library [26] based on power-of-two-choices. All replicas are periodically polled to report their (server-local) RIF. Replica selection is performed by randomly sampling two replicas and selecting the one with lower reported RIF. In our experiments we set the polling interval to 500ms, a 30x faster rate of polling than in the standard YARP-Po2C implementation. We chose the 500ms interval to approximately equalize the total number of RIF reports each client receives, per second, with the number of probe responses received per second by Prequal clients in our ex-

periment.

- Linear, C3, and Prequal all use the asynchronous probing method described in §4, but they differ in the scoring rule used to select a replica from the pool of probe responses.
- Linear uses a linear combination of RIF and latency. To represent RIF and latency in comparable units, we scale RIF by the median query processing time measured on replicas with one request in flight. A replica’s score is defined to be an equally weighted average⁹ of latency and scaled RIF.
- C3 in this paper uses the replica scoring function described in [23] with Prequal’s probing logic. It computes a RIF estimate for each replica as $\hat{q} = 1 + os \cdot n + \bar{q}$, where os is the client-local RIF, n is the number of clients participating in the job, and $\bar{q}$ is an exponentially weighted moving average of the server-local RIF. It then computes a score for each replica as $\Psi = (R - \mu^{-1}) + \hat{q}^3 \cdot \mu^{-1}$, where R and μ^{-1} are exponentially weighted moving averages of the client-local and server-local response time, respectively.
- Prequal uses the HCL replica selection rule described in §4, with the RIF limit quantile set to $Q_{\text{RIF}} = 0.75$.

The results of our experiment are depicted in Fig. 7. We tested the replica selection rules at two different levels of aggregate CPU load — 70% and 90% of our allocation — and reported two quantiles of tail latency, the 90th percentile (p90) and 99th percentile (p99).

The replica selection rules that performed best at all load levels and latency quantiles were C3 and Prequal. What these rules have in common is that they incorporate server-local quantities, they penalize high (server-local) RIF severely, and they favor low-latency replicas when there are multiple replicas with low RIF. In the case of Prequal this behavior is evident in the

⁹In the supplementary material, we report on an experiment showing that the performance of the Linear rule is equally poor for most weightings, except when it degenerates to RIF-only control.

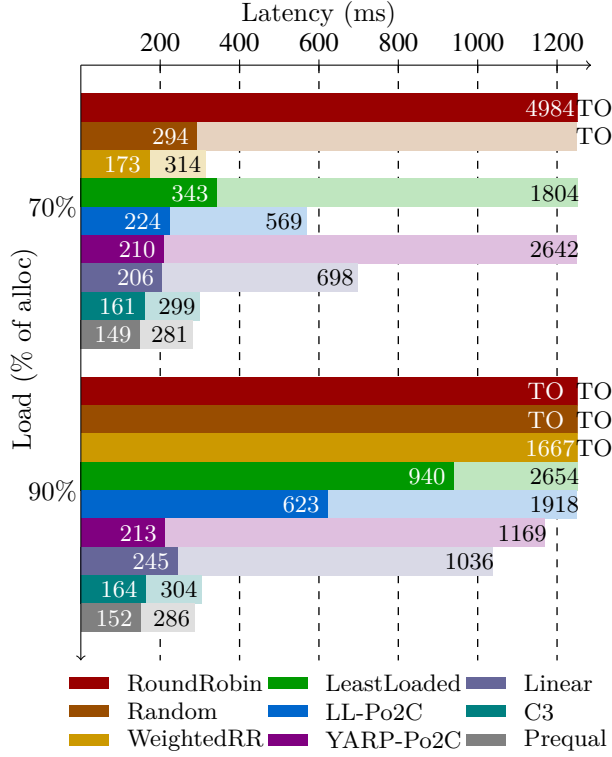


图 7: 对比复制选择规则。条形图中深色部分表示第 90 百分位, 浅色部分表示第 99 百分位。延迟超过 1.25 秒的条形图被截断。超时用 “TO” 表示。

way it distinguishes between hot and cold replicas. In the case of C3 it is implicit in the scoring function’s cubic dependence on estimated queue size, $\hat{q}$: when $\hat{q}$ is near zero it contributes negligibly to the score, but it rapidly grows to predominate the score as $\hat{q}$ moves away from zero.

Comparing C3 and Prequal, one sees a small quantitative advantage for Prequal (3-8%) at all load levels and latency quantiles we tested. Another convenient feature of Prequal’s replica selection rule is that it has a tunable parameter, Q_{RIF} , that allows it to span the full range from RIF-only to latency-only control, depending on the needs of the application. Additionally, Prequal uses fewer signals than C3, simplifying the implementation and monitoring.

It is noteworthy that the LL policy, which bases replica selection on client-local rather than server-local RIF, experiences high p99 latency when load is at 70%

of allocated capacity. Even when a server has no active connections from a given client, it can be highly loaded with queries from other clients. If this possibility happens more than 1% of the time, that is enough to impact p99 latency. When load is 90% of allocation, we see that even p90 latency suffers heavily under the LL policy. Combining LL with Po2C improves latency at all load levels and quantiles in our experiments, but the LL-Po2C rule still lags far behind Prequal and C3.

YARP-Po2C’s selection rule using server-local RIF fares better at high load than LL-Po2C which uses client-local RIF, as one might expect. However, its decisions are often based on stale information due to the infrequent polling of replicas, and this adversely affects latency.

Interestingly, the replica selection rule based on a 50-50 linear combination of latency and RIF performed much worse than Prequal’s and C3’s scoring rules. This indicates that a linear function of RIF doesn’t penalize high RIF severely enough compared to C3’s cubic function or Prequal’s strict prioritization of cold replicas over hot ones.

Finally, the WRR policy performed quite well when load was at 70% of allocation, but its p99 latency suffers greatly at 90% load. This is consistent with the results reported in §5.1, where WRR experiences a very sharp increase in tail latency in response to a modest increase in aggregate CPU load. (WRR experienced this crossover at different load levels in the two experiments, probably because of differing amounts of antagonist load.)

5.3 可调参数

Prequal 具有几个可调节参数, 这些参数会影响其行为和性能。在本节中, 我们评估了其中两个参数——探测速率和 RIF 限制阈值——的影响。前者影响用于副本选择的负载信号的新鲜度, 后者则影响选择规则在将查询导向低估延迟与避免高 RIF 之间的倾向性。

探测速率 在 YouTube 应用中, 探测器相对于查询本身来说非常轻量, 因此探测开销可以忽略不计。不幸的是, 并非所有应用都如此。因此, 我们旨在确定在保留

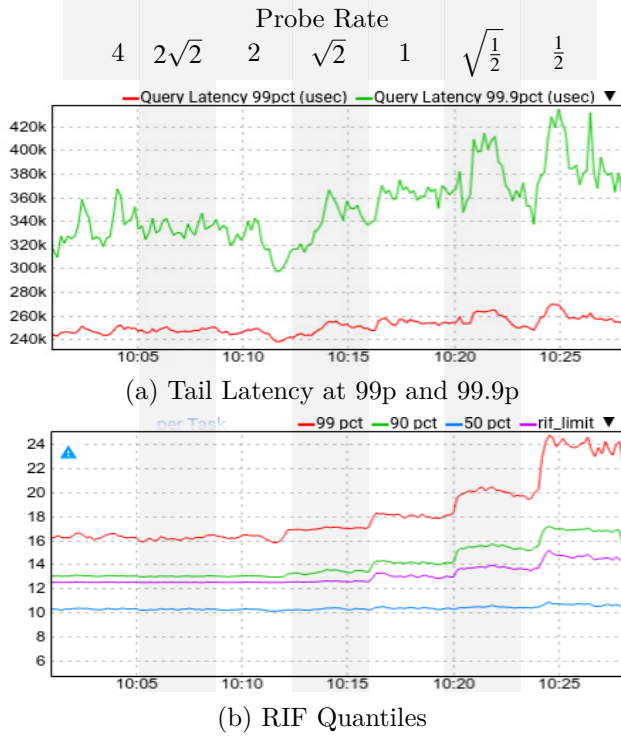


图 8: 探测率实验。速率（上图）以探针数/查询表示。

Prequal 优势的同时，所能容忍的极小探测率。

在本实验中，我们以每步乘法因子 $\sqrt{2}$ 共 6 步，将探测速率从 4 倍查询速率逐步降低至 $\frac{1}{2}x$ 查询速率，同时保持探测移除速率恒定在每次查询 0.25，并通过 (1) 指导使 b_{reuse} 增加以进行补偿。为了放大效果，我们将系统运行得非常热，整体温度约为我们 CPU 分配的 1.5 倍。

Fig. 8 显示了最终的延迟和 RIF，以及 θ_{RIF} 控制参数。核心结论是，当探测速率低于每查询一个探测时，Prequal 对探测速率变得相当敏感，此时负面影响开始显著。在探测速率为 $\frac{1}{\sqrt{2}}x$ 和 $\frac{1}{2}x$ 时，尾部 RIF 分布明显上升，这一变化在两者的延迟分位数中均有体现。据经验观察，我们在多次类似的实验中均发现此现象，且始终集中在每查询约一个探测的水平。

RIF 分位数 我们现在通过一个实验来探究由 HCL 规则中区分热副本与冷副本的分位数所体现的 RIF 与延迟之间的权衡关系。为此，我们将所有编号为奇数的服务器副本 (1、3、...、99) 指定为快速副本，而将所有编号为偶数的副本 (0、2、...、98) 指定为慢速副本。然而，我们并未实际选用不同硬件代际的服务器。相反，在生

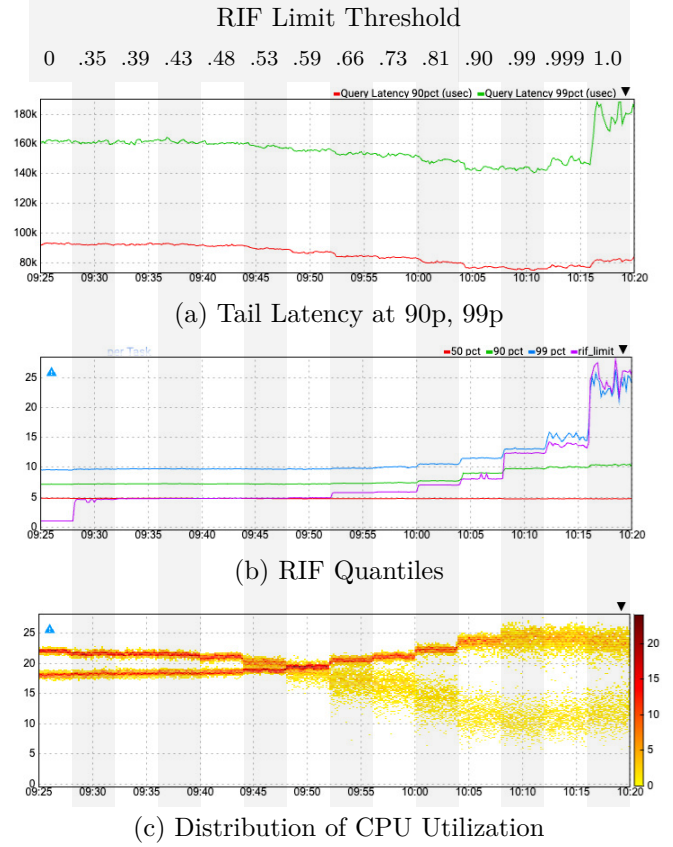


图 9: RIF 限制实验。 Q_{RIF} 从左侧的 0（纯 RIF 控制）变化到右侧的 1（纯延迟控制）。

成查询时，我们人为地将偶数编号副本上的查询工作量放大 2 倍，从而使其消耗 2 倍的 CPU 周期，表现得如同其运行速度慢了 2 倍。此外，我们在整个实验过程中改变 Q_{RIF} 参数：起始值为 0（表示仅采用 RIF 控制），然后以 $\frac{10}{9}$ 为步长，从 $0.9^{10} \approx 0.35$ 逐步增至 0.9；再依次增至 0.99、0.999 和 1（表示仅采用延迟控制）。因此，整个实验共包含 14 个步骤。在本实验全程中，平均负载稳定维持在 CPU 分配量的约 75% 水平。

请注意，Prequal 在最后两个步骤之间的行为实际上存在不连续性。当 $Q_{RIF} = 0.999$ 时，RIF 限制实际上是最大 RIF，这意味着任何与最大值并列的副本都被视为热副本。相比之下，当 $Q_{RIF} = 1$ 时，RIF 限制为 ∞ ，此时每个副本都被视为冷副本。

Fig. 9 展示了结果。RIF 限值阈值 Q_{RIF} 从左侧的 0（纯 RIF 控制）增加到右侧的 1（纯延迟控制）。上方两个图表显示了 p99、p90 和 p50 延迟。正如我们所预期的，随着控制策略逐渐向基于延迟的控制转移，这三

个延迟分位数均下降，直至第 12 步 ($Q_{RIF} = 0.99$)。具体而言，p99 从 $\sim 162\text{ms}$ 下降至 $\sim 142\text{ms}$ (降低 12%)，p90 从 93ms 降至 75ms (降低 19%)，p50 从 $\sim 34.5\text{ms}$ 降至 $\sim 31\text{ms}$ (降低 10%)。在第 13 步时，这三个分位数开始略微上升。当我们在第 14 步切换到完全延迟控制时，所有分位数急剧上升，尤其是 p99，从第 13 步的 $\sim 148\text{ms}$ 上升至约 178ms (上升 20%)。更为显著的是 p99.9 (未显示)，它从第 0 步的 210ms 降至第 12 步的 198ms ，回升至第 13 步的 210ms ，随后在第 14 步出现剧烈波动，范围在 337ms 到 502ms 之间 (相比第 13 步增长 1.6 倍至 2.4 倍)。

看起来，即使是非常微小的 RIF 控制也能产生显著效果。为什么纯粹的延迟控制会导致比 $Q_{RIF} = 0.999$ 更差的延迟呢？我们之前已经暗示了我们的解释：RIF 是负载的一个重要先行信号，因此完全忽略它是不明智的。

底图显示了 CPU 利用率的分布。注意两条交叉的带状区域，它们分别对应于“慢”（即偶数）和“快”（即奇数）副本。“慢”（或“快”）副本对应于向右逐渐降低（或升高）的带状区域。这正是我们所预期的结果，因为增加 Q_{RIF} 意味着我们基于延迟更频繁地进行负载均衡，从而有利于快速副本。此外，当 RIF 控制较高时，CPU 分布更加集中在左侧。这是因为 RIF 是未来 CPU 利用率的一个相当好的预测器。

该实验验证了 HCL 规则，因为将旋钮朝向更多基于延迟的控制确实降低了延迟，即使将其调高至 0.73 (步骤 9)，对 RIF 的影响也微乎其微。在步骤 7 ($Q_{RIF} = 0.59$) 时，所有三个 RIF 分位数的表现与仅使用 RIF 控制时一样好，尽管绝大多数查询都是根据延迟进行路由的。在本实验中，探测池大小为 16，且几乎始终处于满状态。由于采用“选择最佳”和“移除最差”机制，探测池并非均匀随机。但如果它是均匀随机的，那么全部 16 个探测都为热节点的概率大约为 $2^{-16} \approx 1.5 \times 10^{-5}$ ，因为在步骤 7 时， θ_{RIF} 和 p50 RIF 均为 5。否则，查询将根据延迟被路由到冷副本。这使得我们大多数情况下能够兼顾两者：同时基于延迟进行路由，并避免选择最高 RIF 的副本。

6 相关工作

“两个选择的威力”这一随机负载均衡范式首次在 Broder 等人 [2] 和 Mitzenmacher [18] 的具有深远影

响的工作中被分析。这些思想最初源于理论研究，但多年来已广泛应用于工业界的各类场景，正如 2020 年 ACM Paris Kanellakis 理论与实践奖的获奖评语中所指出的 [1]。最初关于两个选择的理论结果针对的是将 n 个球放入 n 个桶的情形，但很快就被推广到“重载”情景，即球的数量超过桶的数量时的情况 [3]。定性地看，这些结果表明，基于随机采样多个选项并选择最优者来实现负载均衡的机制，远优于仅对每个球采样单一桶的简单方法。后续的理论研究进一步证实，这一结论在改变建模假设或采样桶之间的选择规则时仍表现出惊人的鲁棒性；例如参见相关综述 [17, 25]。对于 Prequal 的设计尤为相关的是，该理论被推广至具有记忆的负载均衡过程 [13, 14, 16]，这与我们采用异步探测的用法在理论文献中最接近。

对数据中心任务调度中亚秒级延迟的需求，推动了去中心化负载均衡器的大量研究。该领域的若干系统提出的方法利用了 d -选择 (PodC) 范式的力量。一个典型的例子是 Sparrow [20]，它是一个用于高度并行作业的调度器，每个作业由大量任务组成。Sparrow 的目标是最小化作业响应时间，即作业最后一个组成部分任务完成执行的时间。这一目标促使设计出一套不同的策略，结合批量采样和延迟绑定：一个包含 m 个任务的客户端在其随机选择的服务器副本上放置 $d \cdot m$ 个预留；最早可用的 m 个副本请求运行 m 个任务，其余 $(d-1) \cdot m$ 个预留被取消。这种方法实现了与探测 $d \cdot m$ 个副本并选择其中 m 个最佳响应类似的效果，而预留则起到了探测的作用。我们的场景不同之处在于作业并未进行批量处理，这使我们能够采用更简单且高效的探测机制，避免使用预留和延迟绑定所带来的不期望的开销，这些开销在我们的环境中是不可接受的。基于 Sparrow 的批量采样思想，Eagle [5] 是一种混合调度器，它结合了一个使用批量采样的分布式基于探测的调度器，用于处理短生命周期的任务，同时配备一个集中式调度器，将长期任务分配到副本上。这种混合调度器的设计在作业大小差异较大的情景下具有合理性，因为此时队首阻塞可能成为一个严重问题；而在我们的场景中，作业（查询）都是短生命周期的，因此由 Prequal 提供的完全分布式的负载均衡解决方案在 YouTube 的生产环境以及我们的测试平台上均表现良好。

一个集中式的基于 PodC 的负载均衡器被集成到 RackSched [27] 中，它运行在机架顶部交换机上，以微秒级的粒度做出服务器间调度决策，从而实现机架规模

的计算。在他们的场景中，基于客户端探测的解决方案（如我们的方案）不适用，因为探测的资源开销与服务请求的资源开销相当。取而代之的是，他们的服务器将负载信号附加在正常流量中，调度器将这些负载信号视为隐式探测。另一个使用基于 RIF 的队列策略的机架规模负载均衡系统是 R2P2 [12]，其 JBSQ(n) 策略是对 §5.2 中讨论的 LeastLoaded 策略的一种变体。RackSched 和 R2P2 都依赖所有连接通过单一路由器或交换机；正如 §2 所解释的，在像 YouTube 这样的大规模服务中，使用单一负载均衡器并非可行方案。

C3 [23] 是另一种分布式负载均衡器，其设计目标与 Prequal 相同——即通过根据实时负载反馈自适应地选择副本，以最小化尾部延迟——但采用了截然不同的方法，且需要维护更多的状态。C3 采用分布式速率控制和反压机制，每个客户端均维护针对每个服务器副本的延迟和 RIF 的指数加权移动平均估计值，并对每个服务器副本使用基于 token 桶的速率限制器来。Prequal 的设计中，客户端仅需维护一个大小有界的探测池作为其唯一状态，这种设计更适用于 YouTube 等大规模系统的负载均衡。

许多现有的负载均衡系统（例如，NGINX [19]、Envoy [7]、Finagle [8]、YARP [26]）提供了 PodC 的某种变体，且大多数系统提供（客户端本地）RIF、延迟或两者的组合作为负载均衡信号。如 §5.2 所述，当多个客户端或负载均衡器可能共享同一组副本时，使用客户端或负载均衡器本地的信号会严重影响大规模下的性能。

许多其他生产系统利用 PodC 负载均衡。Netflix 的 Zuul [22] 通过从负载均衡器到后端的去中心化请求路由，随机考虑两个候选服务器，并结合先前响应中的本地和附带的 RIF 统计量来选择负载最低的服务器。与 Prequal 不同，Zuul 避免了主动探测，且不包含延迟信息。NGINX 的实现 [19] 类似于 Zuul，也是无探测的，但其选择准则可配置：“最少连接数”使用来自特定负载均衡器到每个后端的最低 RIF，而“最短时间”则基于先前请求的估计延迟和当前连接数。基于我们对 Prequal 的经验，我们认为 [19, 22] 中描述的负载均衡机制的性能可能通过在附带数据的基础上补充主动探测而得到提升。

致谢

我们感谢当前及前任谷歌同事梅利卡·阿博勒哈萨尼、道格·罗德和阿里克西·坎德拉岑卡，他们曾对 Prequal 的早期原型进行了大量构建与实验。他们先前的辛勤工作使我们在当前项目迭代中获得了巨大的先发优势。我们还从大卫·阿帕特莱奇、托马斯·亚当奇克、普拉蒂克·沃拉和索赫伊尔·叶加内的有益讨论中获益良多，并感谢菲利普·费舍尔-奥格登在阐述方面提出的优秀建议。

参考文献

- [1] 2020 ACM Paris Kanellakis Theory and Practice Award. <https://www.acm.org/media-center/2021/may/technical-awards-2020>, 2020. Accessed: 2023-05-02.
- [2] Yossi Azar, Andrei Z. Broder, Anna R. Karlin, and Eli Upfal. Balanced allocations. *SIAM J. Comput.*, 29(1):180–200, 1999.
- [3] Petra Berenbrink, Artur Czumaj, Angelika Steger, and Berthold Vöcking. Balanced allocations: The heavily loaded case. *SIAM J. Comput.*, 35(6):1350–1385, jun 2006.
- [4] Alejandro Forero Cuervo. Load balancing in the datacenter. In Jennifer Petoff, Niall Murphy, Betsy Beyer, and Chris Jones, editors, *Site Reliability Engineering: How Google Runs Production Systems*, chapter 20, pages 231–247. O'Reilly, Sebastopol, 2016.
- [5] Pamela Delgado, Diego Didona, Florin Dinu, and Willy Zwaenepoel. Job-aware scheduling in eagle: Divide and stick to your probes. In Marcos K. Aguilera, Brian Cooper, and Yanlei Diao, editors, *Proceedings of the Seventh ACM Symposium on Cloud Computing*, Santa Clara, CA, USA, October 5-7, 2016, pages 497–509. ACM, 2016.
- [6] Envoy github repository. <https://github.com/envoyproxy/envoy>. Accessed: 2023-09-21.

- [7] Envoy homepage. <https://www.envoyproxy.io>. Accessed: 2023-09-21.
- [8] Deterministic Aperture: A distributed, load balancing algorithm. https://blog.twitter.com/engineering/en_us/topics/infrastructure/2019/daperture-load-balancer, 2019. Accessed: 2023-09-21.
- [9] gRPC Weighted Round Robin load balancing component. https://github.com/grpc/grpc/tree/master/src/core/ext/filters/client_channel/lb_policy/weighted_round_robin. Accessed: 2023-05-03.
- [10] Chris Jones, John Wilkes, Niall Murphy, and Cody Smith. Service level objectives. In Jennifer Petoff, Niall Murphy, Betsy Beyer, and Chris Jones, editors, *Site Reliability Engineering: How Google Runs Production Systems*, chapter 4, pages 37–48. O’Reilly, Sebastopol, 2016.
- [11] Manolis Katevenis, Stefanos Sidiropoulos, and Costas Courcoubetis. Weighted round-robin cell multiplexing in a general-purpose ATM switch chip. *IEEE Journal on Selected Areas in Communications*, 9(8):1265–1279, 1991.
- [12] Marios Kogias, George Prekas, Adrien Ghosn, Jonas Fietz, and Edouard Bugnion. {R2P2}: Making {RPCs} first-class datacenter citizens. In 2019 USENIX Annual Technical Conference (USENIX ATC 19), pages 863–880, 2019.
- [13] Dimitrios Los, Thomas Sauerwald, and John Sylvester. Balanced allocations: Caching and packing, twinning and thinning. In *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 1847–1874, 2022.
- [14] Dimitrios Los, Thomas Sauerwald, and John Sylvester. Balanced allocations with heterogeneous bins: The power of memory. In *Proceedings of the 2023 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 4448–4477, 2023.
- [15] Michael Mitzenmacher. How useful is old information? *IEEE Trans. Parallel Distributed Syst.*, 11(1):6–20, 2000.
- [16] Michael Mitzenmacher, Balaji Prabhakar, and Devavrat Shah. Load balancing with memory. In *The 43rd Annual IEEE Symposium on Foundations of Computer Science, 2002. Proceedings.*, pages 799–808. IEEE, 2002.
- [17] Michael Mitzenmacher, Andréa W. Richa, and Ramesh Sitaraman. The power of two random choices: A survey of techniques and results. In Sanguthevar Rajasekaran, Panos M. Pardalos, and José Rolim, editors, *Handbook of Randomized Computing*, volume 1, pages 255–312. Springer Science & Business Media, 2001.
- [18] Michael David Mitzenmacher. *The Power of Two Choices in Randomized Load Balancing*. PhD thesis, UC Berkeley, 1996.
- [19] Nginx and the “Power of Two Choices” load-balancing algorithm. <https://www.nginx.com/blog/nginx-power-of-two-choices-load-balancing-algorithm/>, 2018. Accessed: 2023-05-04.
- [20] Kay Ousterhout, Patrick Wendell, Matei Zaharia, and Ion Stoica. Sparrow: Distributed, low latency scheduling. In *Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP)*, pages 69–84, 2013.
- [21] Gahyun Park. A generalization of multiple choice balls-into-bins. In *Proceedings of the 30th Annual ACM SIGACT-SIGOPS Symposium on Principles of Distributed Computing*, pages 297–298, 2011.
- [22] Rethinking Netflix’s edge load balancing. <https://netflixtechblog.com/netflix-edge-load-balancing-695308b5548c>, 2018. Accessed: 2023-05-04.
- [23] Lalith Suresh, Marco Canini, Stefan Schmid, and Anja Feldmann. C3: Cutting tail latency in cloud data stores via adaptive replica selection. In *12th*

USENIX Symposium on Networked Systems Design and Implementation (NSDI 2015), pages 513–527, 2015.

- [24] Varun Talwar. gRPC: a true internet-scale RPC framework is now 1.0 and ready for production deployments. <https://cloud.google.com/blog/products/gcp/grpc-a-true-internet-scale-rpc-framework-is-now-1-and-ready-for-production-deployments>, 2016. Accessed: 2023-09-21.
- [25] Udi Wieder. Hashing, load balancing and multiple choice. *Foundations and Trends® in Theoretical Computer Science*, 12(3–4):275–379, 2017.
- [26] YARP: Yet Another Reverse Proxy. <https://microsoft.github.io/reverse-proxy/>. Accessed: 2023-09-21.
- [27] Hang Zhu, Kostis Kaffes, Zixu Chen, Zhenming Liu, Christos Kozyrakis, Ion Stoica, and Xin Jin. Racksched: A microsecond-scale scheduler for rack-scale computers. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020*, pages 1225–1240. USENIX Association, 2020.

A 延迟与 RIF 的线性组合

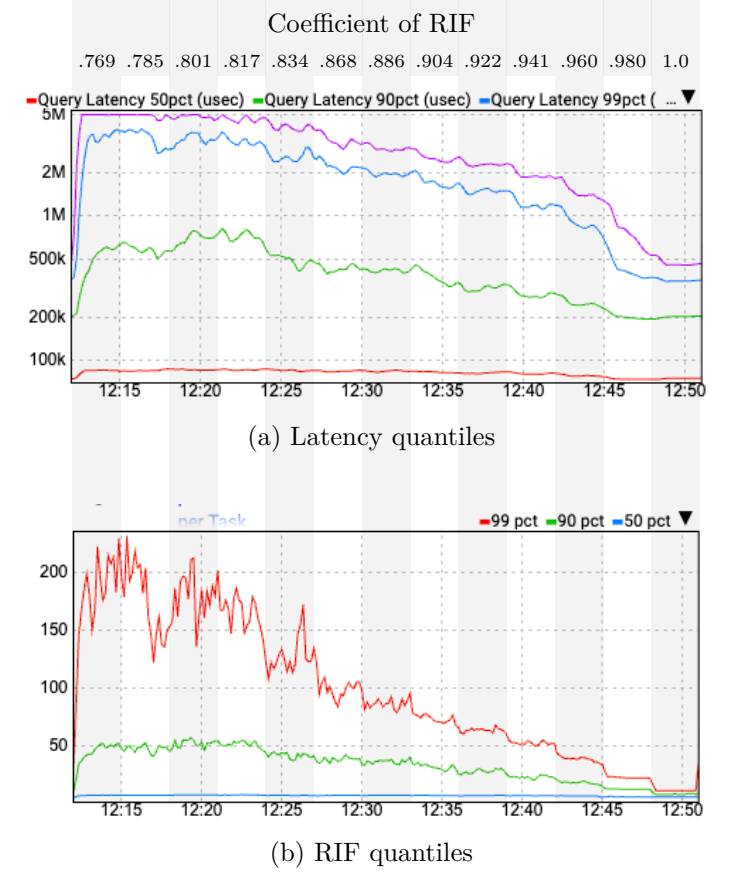


图 10: 基于延迟与 RIF 的线性组合评估副本选择规则。

为了评估使用延迟和 RIF 线性组合的副本选择规则的有效性，我们使用了测试平台。¹⁰ 尝试使用一种变体的 Prequal，该变体采用如 §4 所述的异步探测方法，并对其进行修改，通过延迟和 RIF 的线性组合来选择副本。换句话说，HCL 副本选择规则被替换为一种策略，即在探测池中表示的副本之间，通过最小化得分来选择。

$$\text{score}_i^\lambda = (1 - \lambda) \cdot \text{latency}_i + \lambda \cdot \alpha \cdot \text{RIF}_i \quad (2)$$

其中， latency_i , RIF_i 分别表示从副本 i 发出的探测响应中的延迟和 RIF， α 是应用于 RIF 的缩放因子，用于将其转换为与延迟相同单位， $\lambda \in [0, 1]$ 是可调参数，用于调整对延迟和 RIF 的相对权重。设置 $\lambda = 0$ 对应仅延迟控制，而 $\lambda = 1$ 对应仅 RIF 控制。

为了设置比例因子 α ，我们使用了在飞行中只有一个请求的服务器副本的近似中位数查询响应时间。该值

¹⁰ 参见 §5 开头的测试环境描述。

最终确定为 75 毫秒。注意, 对于任意 $\alpha > 0$, 通过在区间 $[0, 1]$ 内变化 λ 所获得的评分规则集 $\{\text{score}^\lambda \mid 0 \leq \lambda \leq 1\}$ 始终等于延迟与 RIF 的凸组合集合。换句话说, 我们对 $\alpha = 75\text{ms}$ 的选择仅影响该评分规则集由 λ 参数化的形式, 而不影响该集合本身。

我们通过测量延迟和 RIF 的分位数来评估线性组合评分规则, 在测试环境中保持聚合 CPU 利用率恒定 (等于我们分配的 94%), 同时改变 λ 。副本被划分为数量相等的快速 (奇数编号) 和慢速 (偶数编号) 副本, 查询处理速度相差 2 倍, 与 §5.3 中描述的 RIF 分位数实验一致。我们最初尝试在 $[0, 1]$ 的全范围内以 0.1 为增量变化 λ 。从该初步实验中明显看出, 线性组合规则中 $\lambda \leq 0.7$ 的表现远差于较大的 λ 值, 这促使我们以更高分辨率考察 Fig. 10 所示范围。在本实验中测试的 13 种线性组合中, 所有延迟和 RIF 的分位数均随着 λ 的增加而单调改善, 其中 $\lambda = 1$ (即仅使用 RIF 的控制) 在大多数情况下以显著优势优于其他线性组合反馈控制规则。

回顾 Fig. 9 在 §5.3 中的内容可知, 仅使用 RIF 的控制策略 (在该图中由 RIF 限值阈值 1.0 表示, 即最右侧的配置) 在所有延迟和 RIF 的分位数上均表现得更差于 Prequal。由于此处报告的实验结果表明, 仅使用 RIF 的控制策略在所有情况下均严格优于其他任意延迟与 RIF 的线性组合, 因此根据传递性可得出结论: Prequal 严格支配所有延迟与 RIF 的线性组合。